HANFIS: A Hierarchical Adaptive Neuro-Fuzzy Inference System for Evidence-Based Dynamic Cybersecurity Risk Scoring in Cloud-Native Kubernetes Deployments

Mohammad Hafiz Hersyah[1*], Derisma Derisma[2] and Budi Rahmadya[1]

[1*]Computer Engineering Department, Andalas University, Kampus Limau Manis Street, Padang, 25162, West Sumatera, Indonesia.
[2]Informatics Department, Andalas University, Kampus Limau Manis Street, Padang, 25162, West Sumatera, Indonesia.

*Corresponding author(s). E-mail(s): mohammadhafizhersyah02@it.unand.ac.id;
Contributing authors: derisma@it.unand.ac.id; budi-r@it.unand.ac.id;

Abstract

Static, expert-weighted fuzzy risk scores go stale between elicitations, while a flat Adaptive Neuro-Fuzzy Inference System (ANFIS) over the seven inputs relevant to cloud-native risk – CVSS, exploitability, patch risk, MFA risk, Kubernetes exposure, asset criticality, and time-varying threat intelligence – requires $\mathbf{2^7 = 128}$ rules and 1,052 parameters under exhaustive grid partitioning. We propose HANFIS, a domain-informed hierarchical ANFIS that separates vulnerability and environmental reasoning into two first-tier Sugeno subsystems before an integration tier combines them with dynamic threat intelligence, cutting the rule base to 34 rules and 176 parameters (a 73.4% and 83.3% reduction, respectively) while keeping every intermediate variable semantically interpretable. We construct a temporal exploitability-exposure target from sealed weekly EPSS and CISA KEV snapshots over a 57-week window and a digest-pinned container-image inventory, validated through a data-quality gate (6,357 labelled asset-weeks from 120 assets at the time of writing, non-missing rate 0.93, zero unresolved evidence rows) and evaluated with a grouped, embargoed temporal split so that no image or outcome window leaks across folds. On this real-world inventory, a naive 4-week persistence baseline already explains most of the label's variance

and a single CVSS score explains almost none, setting a high and non-obvious bar for any learned model. Under a protocol frozen and dated before training began, HANFIS reduces mean squared error by 96.7% against a single CVSS score, 69.6% against a linear model over the same seven inputs, and 66.5% against a parameter-matched flat ANFIS initialised by subtractive clustering (all three 95% bootstrap confidence intervals excluding zero); HANFIS is not distinguishable from the persistence baseline overall (confidence interval crossing zero), and a post hoc analysis shows it is better only around structural breaks in the label and worse in stable periods, so we report H1b as unconfirmed rather than as a tie. The hybrid least-squares/gradient training scheme and its monotonicity and membership-function ablations are reported alongside these confirmatory comparisons, and a Fuzzyfortify comparison is not claimed because its native inputs are not available for the present assets.

Keywords: Adaptive Neuro-Fuzzy Inference System, Nonlinear Risk Mitigation, Security Risk Assessment, Container Orchestration, Cloud-Native Security

1 Introduction

Cloud-native computing has changed both the scale and the temporal behaviour of enterprise attack surfaces. Applications are increasingly decomposed into loosely coupled microservices and deployed through container orchestration, while Kubernetes Pods are intentionally treated as relatively ephemeral entities that can be created, replaced, rescheduled, or terminated as operating conditions change [1, 2]. These characteristics improve elasticity and operational agility, but they also reduce the useful lifetime of security assessments derived from periodic observations. A risk value calculated before a deployment, configuration change, or new exploitation signal may no longer represent the state of the workload when a security decision is required. This problem is reinforced by Zero Trust Architecture (ZTA), which removes implicit trust based on network location and requires access decisions to consider the current identity, resource, and operational context [3, 4]. Consequently, cloud-native risk assessment must become sufficiently adaptive to follow changes in vulnerability condition, workload exposure, authentication posture, asset importance, and external threat activity.

Our previous Fuzzyfortify framework [5] established an interpretable foundation for cybersecurity risk assessment by using fuzzy multi-criteria reasoning to represent expert knowledge and linguistic uncertainty. This approach remains valuable when observations are incomplete and organizational priorities cannot be expressed adequately through crisp thresholds. Its principal limitation, however, is temporal rather than representational: expert-derived weights and rules remain fixed until they are elicited or revised again. They do not learn from accumulated operational evidence, and a score generated from a static configuration may therefore remain unchanged even when telemetry indicates active exploitation, lateral movement, loss of multifactor authentication protection, increased Kubernetes exposure, or delayed remediation.

The resulting research gap is not simply a conflict between subjective and objective assessment. It is the absence of a reproducible mechanism that preserves expert interpretability while adapting the input–output relationship from observed data and continuously recalculating risk from the current system state.

An Adaptive Neuro-Fuzzy Inference System (ANFIS) offers a suitable basis for addressing this gap because it combines human-readable fuzzy rules with data-driven adjustment of membership and consequent parameters [6]. A conventional flat ANFIS, however, becomes increasingly difficult to train and inspect as its input dimension grows. Under exhaustive grid partitioning, a model with n inputs and m membership functions per input may require m^n fuzzy rules. For the seven inputs considered in this study, an exhaustive flat partition with two membership functions per input already requires $2^7 = 128$ rules before any pruning. Such growth increases the number of adaptive parameters, the least-squares problem size, memory consumption, and inference operations. These costs conflict with the objective of executing risk inference close to cloud-native workloads on resource-constrained edge nodes.

This study, therefore, proposes a domain-informed Hierarchical Adaptive Neuro-Fuzzy Inference System (HANFIS). The first tier contains two parallel subsystems. The vulnerability subsystem combines the CVSS Base Score, Exploitability, and Patch Risk to produce a Vulnerability Risk Index. The environmental subsystem combines MFA Risk, Kubernetes Exposure, and Asset Criticality to produce an Environmental Risk Index. A second-tier integration subsystem combines these two intermediate indices with a dynamic Threat Intelligence input to produce the final Dynamic Risk Score. This organization is consistent with established risk-assessment logic, in which vulnerability and threat information are interpreted within an organizational context [7]. It also prevents threat information from being counted twice by separating the CVSS Base Score from the dynamic threat signal, in accordance with the distinction among Base, Threat, and Environmental metric groups in CVSS v4.0 [8]. Threat Intelligence can be operationalized through time-varying evidence such as EPSS probability, known exploitation status, exploitation recency, and source confidence; EPSS, in particular, estimates the probability that a published vulnerability will be exploited in the following 30 days [9].

The hierarchical decomposition is not merely a conceptual rearrangement. If every input uses m membership functions, the exhaustive rule count is reduced from

$$R_{\text{flat}} = m^7 \tag{1}$$

to

$$R_{\text{HANFIS}} = 2m^3 + m_I^2\, m, \tag{2}$$

where m is the number of membership functions on each of the seven raw inputs and m_I is the number of membership functions on the two intermediate variables v and e inside the integration tier (Section 3.2). With the frozen configuration $m = 2$, $m_I = 3$ (amendment A-02), the proposed structure requires 34 candidate rules rather than 128, a 73.4% reduction; the corresponding parameter count falls from 1,052 to 176, an 83.3% reduction (Table 1). Hierarchical fuzzy systems have been shown to mitigate dimensionality-driven rule growth while retaining nonlinear approximation capability [10, 11]. Nevertheless, a lower theoretical rule count does not by itself establish

superiority. Hierarchical compression may discard cross-subsystem interactions, and the sequential integration tier introduces an additional inference stage. The proposed architecture must therefore be compared against Fuzzyfortify, a flat grid-partitioned ANFIS, and a rule-reduced or clustering-based flat ANFIS using identical observations, data partitions, and evaluation conditions.

Placing inference at the edge introduces a second research problem. Cloud-native security monitoring requires authentication, authorization, service visibility, and continuous observation across a large number of interacting components [1]. Transferring all telemetry to a centralized decision service can add communication overhead, create dependence on network availability, and separate the decision point from the workload requiring protection. An edge-deployable HANFIS could reduce this separation, but only if its training and inference costs are demonstrably compatible with the available processor, memory, and energy budget. Accordingly, computational optimization is treated as a measurable research objective rather than an assumed benefit. The evaluation includes rule count, trainable parameters, training duration, peak memory, throughput, and median and tail inference latency, with edge-platform measurements left to future work.

A dynamic score is operationally useful only when it can support a controlled security decision. The proposed framework therefore specifies a policy-translation layer, designed but not evaluated in this paper, that maps validated risk states to predefined responses, such as generating an alert, restricting communication, pausing a container, or isolating a compromised workload. This layer is designed as an auditable policy enforcement mechanism rather than an unconstrained autonomous controller. Critical safeguards include monotonicity checks, explicit high-risk override rules, confidence thresholds, response logging, and a configurable requirement for human authorization for disruptive actions. In this way, the framework is designed to connect adaptive assessment to measurable time-to-mitigation in future work, while preserving accountability and operational safety.

The aim of this study is to design, implement, and evaluate a HANFIS with a reduced rule base and intermediate variables that are interpretable by construction, for dynamic cybersecurity risk assessment in cloud-native environments. The study investigates whether domain-informed hierarchical decomposition can preserve or improve risk-estimation performance while substantially reducing the rule base and parameter count of a flat ANFIS; policy support and edge execution are designed but left to future work.

The principal contributions of this study are stated as follows:

C1. A domain-informed HANFIS architecture that separates vulnerability and environmental reasoning before integrating their outputs with dynamic threat intelligence to produce a continuously recalculated Dynamic Risk Score.

C2. A computational optimization strategy that changes exhaustive rule growth from m^7 to $2m^3 + m_I^2 m$, accompanied by empirical measurements of parameter count, training cost, memory consumption, throughput, and inference latency.

C3. An operational definition of dynamic cybersecurity risk that combines CVSS Base severity, patch condition, MFA weakness, Kubernetes exposure, asset criticality, and time-varying threat evidence without duplicating CVSS threat metrics.

C4. An auditable policy-translation mechanism (designed but not evaluated in this paper) that converts validated risk states into bounded response recommendations with monotonicity constraints, critical-risk overrides, confidence controls, and an execution log.

C5. A reproducible evaluation against persistence, single-CVSS, ridge and flat ANFIS baselines (full and parameter-matched) using predictive and computational metrics under common grouped, embargoed temporal partitions; a Fuzzyfortify comparison is not claimed (Section 4.3).

The empirical analysis is guided by the following hypotheses:

H1a: Improvement over static expert scoring. HANFIS will produce significantly lower risk-estimation error than a single CVSS score, the current-practice baseline.

H1b: Improvement over naive persistence. HANFIS will reduce mean squared error relative to a causal 4-week persistence forecast by at least a smallest-effect-size-of-interest of 5%, with the lower bound of its 95% confidence interval above zero. Persistence, not a single CVSS score, is the empirically strongest baseline observed on the pilot dataset (Section 4.3) and is therefore the bar HANFIS must clear, not an optional comparator.

H1c: Value of hierarchical structure over a linear model. HANFIS will be non-inferior to a ridge-regularised linear model on the same seven inputs, with superiority reported when the confidence interval excludes zero.

H2: Computational efficiency. For an equivalent input set and membership-function configuration, HANFIS will require significantly fewer fuzzy rules and adaptive parameters and will reduce training time, peak memory, and inference latency relative to flat grid-partitioned ANFIS.

H3: Performance preservation. The predictive performance of HANFIS will be non-inferior to a parameter-matched flat ANFIS (FlatANFISMatched, ±5% parameters) within a prespecified margin, despite the reduction in rule-base complexity. A flat ANFIS's plateau below the full-capacity reference is treated as evidence for the mechanism this decomposition trades away – cross-subsystem interaction discarded by the scalar inter-tier bottleneck – rather than as a falsification of H2/H3 by itself.

H4: Threat responsiveness. Including time-varying Threat Intelligence will improve the prioritization of actively exploited or high-probability vulnerabilities compared with an otherwise identical HANFIS without this input.

H5 (edge execution), originally framed as a fifth hypothesis, is demoted to future work: the Jetson Orin Nano node in the present testbed is a telemetry client only, and HANFIS inference runs cluster-side, so no edge-execution timing evidence exists to test it against. H1a–H1c and H2–H3 are pre-registered against the frozen protocol, seed list, and data split described in Section 4.2; results are reported in Section 4.3 (Table 5).

Together, these contributions position HANFIS as more than a higher-capacity replacement for static fuzzy assessment. The proposed model is intended to provide a traceable bridge from expert knowledge to data-adaptive inference, from high-dimensional risk factors to a computationally tractable rule base, and, as designed but

not yet evaluated extensions, from risk estimation to bounded operational response and to edge execution.

2 Literature Review

2.1 ANFIS

ANFIS is a hybrid algorithm that integrates the learning ability of ANN into the FIS design procedure. ANFIS uses the existing input–output data pairs to tune FIS trainable parameters and generate IF–THEN fuzzy rules. ANFIS architecture consists of antecedent and consequent parts whose parameters are connected by fuzzy rules based on Sugeno and Tsukamoto-type fuzzy models. The corresponding fuzzy rules with two inputs x_1 and x_2 and one linear output function (f), are given as:

Rule # 1. IF x_1 is A_1 AND x_2 is B_1 THEN

$$f_1 = p_1 \cdot x_1 + q_1 \cdot x_2 + r_1$$

Rule # 2. IF x_1 is A_2 AND x_2 is B_2 THEN

$$f_2 = p_2 \cdot x_1 + q_2 \cdot x_2 + r_2$$

The term "hierarchical ANFIS" and the acronym HANFIS are not introduced here for the first time: a prior hierarchical ANFIS grouping vulnerability-adjacent features has reportedly been published under this name (Soozanchi et al.), though we were unable to independently verify or cite the specific source; and the present work's contribution is the specific domain decomposition (vulnerability/environmental/threat tiers), the temporal, evidence-sealed target definition of Section 3.5, and the accompanying real-world evaluation protocol, not the term itself. Where the distinction matters we refer to the present architecture as HANFIS-DRS (Dynamic Risk Scoring) to avoid conflating the two.

3 Methodology

3.1 HANFIS Architecture and Risk Formulation

This study adopts a controlled, repeated-measures experimental design to develop and validate a telemetry-aware Hierarchical Adaptive Neuro-Fuzzy Inference System (HANFIS) for dynamic risk scoring in a cloud-native environment. The methodological premise is that runtime telemetry cannot be incorporated into a cybersecurity risk model without first establishing its provenance, measurement cost, stability, and reproducibility. Otherwise, resource consumption or latency introduced by the observability stack itself could be misinterpreted as a change in workload exposure and, consequently, as a change in cyber risk.

The study therefore separates two related but analytically distinct objectives. The first objective is to quantify the overhead associated with Kubernetes observability. The second is to transform validated runtime measurements into features for the

environmental subsystem of HANFIS. The overhead experiment compares the same baseline workload under two conditions: monitoring enabled (ON) and monitoring disabled (OFF). The workload, service configuration, request path, cluster topology, sampling procedure, and validation gates are held constant. Monitoring state is the principal experimental factor. This design provides a control condition against which the incremental cost of telemetry collection can be estimated.

The observability experiment comprises ten paired runs (Runs 7–16). Conditions are counterbalanced across runs using ON–OFF and OFF–ON sequences. Counterbalancing reduces systematic error associated with execution order, cache warming, temporal load variation, and cluster stabilisation. Each transition is guarded by a preflight or post-transition validation step. A condition is admitted to measurement only when the Kubernetes nodes, baseline workload, HTTP service, and required monitoring state satisfy predetermined acceptance criteria. Failed gates produce a `HOLD` decision rather than an experimental observation. This prevents measurements obtained from a degraded or incorrectly configured system from entering the analytical dataset.

3.2 HANFIS Architecture, Rule Count, and Parameter Complexity

HANFIS composes three first-order Sugeno ANFIS panels. The vulnerability panel $V(x_1, x_2, x_3) \to v$ takes CVSS, Exploitability, and Patch Risk; the environmental panel $E(x_4, x_5, x_6) \to e$ takes MFA Risk, Kubernetes Exposure, and Asset Criticality; the integration panel $I(v, e, x_7) \to \tilde{r}$ takes the two intermediate indices and Threat Intelligence, with the final score $r = \mathrm{clip}(\tilde{r}, 0, 1)$. No panel is given an intermediate target: v and e are latent, and gradients flow end-to-end from the loss on r back through both tiers.

Each panel uses Gaussian membership functions,

$$\mu_{ij}(z_i) = \exp\left(-\frac{(z_i - c_{ij})^2}{2\sigma_{ij}^2}\right), \qquad i = 1, \ldots, n,\ j = 1, \ldots, m, \tag{3}$$

Sugeno-normalised firing strengths $\bar{w}_k = w_k / \sum_l w_l$ with $w_k = \prod_i \mu_{i,j_k(i)}(z_i)$, and first-order consequents $f_k(z) = \beta_{k0} + \sum_i \beta_{ki} z_i$, so that a panel's output is $\sum_k \bar{w}_k f_k(z)$. The raw inputs $x_1, \ldots, x_7$ each use $m = 2$ membership functions; the intermediate variables v, e inside the integration panel use $m_I = 3$ (amendment A-02), which turns v and e into the linguistic categories {low, medium, high} rather than a single coarse split, following the requirement that reduced intermediate variables remain semantically interpretable (Lee, Chung, and Yu [12]). A connected trapezoidal (Ruspini) membership family is available as an ablation alternative to the free Gaussian family (Section 4.5).

Table 1 reports the resulting rule and parameter counts against a full flat ANFIS over the same seven inputs with $m = 2$ membership functions each. The hierarchy reduces the rule count by 73.4% and the parameter count by 83.3%; unlike the flat

model, rule growth is linear in the number of panels rather than exponential in the number of inputs [10, 13].

Table 1 Rule and parameter counts by panel ($m = 2$ on raw inputs, $m_I = 3$ on v, e) against a full flat ANFIS over the same seven inputs.

Panel	Inputs	Rules	Premise params.	Consequent params.	Total
V	x_1, x_2, x_3	8	12	32	44
E	x_4, x_5, x_6	8	12	32	44
I	v, e, x_7	18	16	72	88
HANFIS (total)	7 raw + 2 intermediate	**34**	40	136	**176**
Flat ANFIS (reference)	7 raw	128	28	1,024	1,052

Consequent slopes in every panel are constrained non-negative through a softplus reparameterisation (amendment A-04), so that each rule's consequent is non-decreasing in every input under the stated higher-is-riskier convention; intercepts stay free and rule weights are normalised, so r itself is not guaranteed to be monotone, and monotonicity is therefore checked empirically (Section 4.7). Training is hybrid (amendment A-06): the integration panel's consequents are solved in closed form by ridge least squares every epoch,

$$\beta = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top y, \qquad \lambda \geq 10^{-3}, \tag{4}$$

with Φ the per-rule design matrix $\bar{w}_k \cdot (1, z_1, \ldots, z_n)$, while premise parameters are updated by Adam. An alternating variant (E2) additionally solves both lower panels' consequents in closed form every epoch, by freezing the integration panel's firing weights and consequents so that the final output becomes affine in v and e per row, which in turn makes it linear in the lower panels' own consequents given their (frozen-premise) firing weights; this reduces to Adam training premise parameters only. Three pre-registered restarts with fixed seeds are trained per fold, patience 100 on validation loss with a minimum of 200 epochs, and the restart with the lowest validation loss is kept (amendment A-09); a pure end-to-end Adam variant without any closed-form step is retained as an ablation (Section 4.5) because it is unstable on some seeds without the closed-form anchor.

3.3 Experimental Platform and Telemetry Validation

The experimental workflow was implemented in Ubuntu 24.04 under Windows Subsystem for Linux 2 (WSL 2), with Visual Studio Code connected directly to the WSL environment. Docker Desktop provided the container runtime and Kubernetes integration. The research workspace was maintained under a dedicated project hierarchy, while shell procedures were stored as versionable `.sh` files rather than being executed only as transient terminal commands. This decision improved procedural transparency,

reduced copy-and-paste errors, and allowed every experimental gate to be re-examined without reconstructing the original command sequence.

The Kubernetes testbed comprised two Ready nodes and a baseline application, denoted P0, deployed with two ready replicas in the `hanfis-research` namespace. A load-generator pod accessed P0 through an internal Kubernetes service. Service availability was verified using an HTTP request from inside the cluster; status code 200 was required at each critical gate. The observability stack was deployed in a separate `monitoring` namespace and consisted of Prometheus, Prometheus Operator, kube-state-metrics, and node-exporter components. In its validated ON state, the stack contained five fully ready and running monitoring pods, including node-level exporters. In the OFF condition, the deployments and stateful workload were scaled to zero and the exporter DaemonSet was made unschedulable through a controlled node-selector patch. The original replica counts and Kubernetes object definitions were saved before the transition so that the monitoring state could be restored deterministically.

System preparation was organised as a sequence of auditable gates. The gates cover the baseline workload, cluster health, the observability deployment, the measurement procedure, protocol freezing and the generation of SHA-256 manifests; the criterion applied at each gate is listed in Table 2. The first gates of the protocol-development phase (O5–B4.10 to O5–B4.11B) are kept in the repository as scripts and describe the procedure only; since one of their gate reports could not be reconstructed after the fact, they are not cited as evidence of passing. The evidence that the gates were satisfied is the set of run archives and manifests for Runs 7–16 (Section 3.4), in each of which both conditions were admitted only after the gate criteria held. Because P0 stayed at two of two ready replicas with HTTP status 200 across every admitted transition, the change in observability state did not itself interrupt the target service.

3.4 Observability-Overhead Protocol (external-v3.0)

The gates and integrity controls of Table 2 were developed during the first runs and were carried forward unchanged into protocol `external-v3.0`, which produced all observability-overhead results reported in Section 4.1. The protocol comprises ten complete counterbalanced ON–OFF pairs (Runs 7–16): Runs 8, 10, 12, 14 and 16 were executed OFF–ON, and Runs 7, 9, 11, 13 and 15 ON–OFF. The earlier Runs 1–6 belong to the protocol-development phase and are not pooled with the ten pairs. One aborted attempt of Run 7 and one aborted attempt of Run 17 failed the integrity gate and are recorded as incidents rather than observations.

The protocol follows a fail-closed rule: a condition is admitted to measurement only when the nodes, the P0 workload (two of two replicas), the HTTP service (status 200) and the required monitoring state satisfy the acceptance criteria of Table 2; a failed gate yields a `HOLD` decision, not an observation. Each run is archived with a SHA-256 manifest, so that the ten run archives and the cumulative analysis can be re-verified without reconstructing the original command sequence.

Telemetry preserves the link between each value and its Kubernetes object, node, container, condition and run. The measurements are node-, pod- and container-level CPU and memory, pod readiness, HTTP outcome and response latency; identical

Table 2 Experimental preparation, acceptance criteria, and methodological purpose.

Preparation element	**Acceptance criterion**	**Methodological purpose**	**Evidence (Runs 7–16)**
Cluster health	Two Kubernetes nodes in Ready state	Excludes measurements obtained during node degradation or loss of schedulability	Gate passed in all ten pairs
Baseline workload P0	Two of two replicas Ready	Holds application capacity constant across monitoring conditions	Gate passed in all ten pairs
Service health	Internal HTTP response code 200	Confirms that the measurement target remains operational before and after state transitions	Gate passed in all ten pairs
Monitoring ON state	Five of five pods Running and fully Ready; Prometheus 1/1 Ready	Defines a complete observability treatment rather than a partially deployed stack	Gate passed in all ten pairs
Monitoring OFF state	Zero monitoring pods remaining	Defines a valid control condition without residual collector workloads	Gate passed in all ten pairs
Protocol freeze	Scripts, controller definitions, services, system fingerprint, and replication plan present	Prevents unrecorded procedural drift between runs	Applied to Runs 7–16; two aborted attempts logged as incidents
Integrity control	SHA-256 manifests validate without mismatch	Detects post hoc alteration or corruption of experimental artefacts	SHA-256 manifest retained for each of the ten run archives
Counterbalancing	Prespecified ON–OFF or OFF–ON order	Reduces order, warm-up, cache, and time-dependent bias	Five ON–OFF and five OFF–ON pairs

request-generation and sampling procedures are applied in the ON and OFF conditions. Configuration files, event logs, gate reports and manifests document how each observation was produced and are not treated as HANFIS training features.

Let $Y_{r,c,k}$ denote observation k of run r under condition $c \in \{\mathrm{ON}, \mathrm{OFF}\}$, where Y may be CPU, memory or latency. The run mean is

$$\bar{Y}_{r,c} = \frac{1}{n_{r,c}} \sum_{k=1}^{n_{r,c}} Y_{r,c,k}, \tag{5}$$

the paired absolute overhead is $\Delta Y_r = \bar{Y}_{r,\mathrm{ON}} - \bar{Y}_{r,\mathrm{OFF}}$, and the relative overhead, when the OFF denominator is non-zero, is

$$\Delta Y_r^{(\%)} = \frac{\bar{Y}_{r,\mathrm{ON}} - \bar{Y}_{r,\mathrm{OFF}}}{\bar{Y}_{r,\mathrm{OFF}}} \times 100\%. \tag{6}$$

The run, not the individual sample, is the independent replication unit, which avoids pseudoreplication from temporally adjacent samples. Inference over the ten paired differences uses an exact two-sided sign-flip test, whose smallest attainable p with ten pairs is $2/2^{10} = 0.002$, together with a 10,000-resample bootstrap confidence interval (fixed seed) for the mean difference. Effect sizes, confidence intervals and the consistency of direction carry more interpretive weight than a binary significance decision. The estimated increment is an instrumentation cost; it must not be read as a change in application exposure.

3.5 Realised Exploitability-Exposure Target and Real-World Asset-Week Dataset

The Kubernetes-exposure telemetry described above feeds one environmental input; the label used to train and evaluate HANFIS is constructed separately, from realised vulnerability outcomes rather than expert judgement. The observation unit is an asset-week: one digest-pinned container image a observed at one Monday t. For a 30-day horizon $\Delta = 30$ discretised into four weekly checkpoints $T = \{t{+}7, t{+}14, t{+}21, t{+}28\}$,

$$y(a,t) = \frac{100}{|T|} \sum_{\tau \in T} \max_{v} \left[X_v(\tau)\, S_{v,a}(\tau) \right], \tag{7}$$

$$\begin{aligned} X_v(\tau) &= \max\left(\mathrm{EPSS}_v(\tau), \mathbb{1}[v \in \mathrm{KEV}(\tau)]\right), \\ S_{v,a}(\tau) &= \rho_{v,a} \cdot \mathbb{1}[v \text{ affected and reachable on } a \text{ at } \tau], \end{aligned} \tag{8}$$

where $\rho_{v,a} \in \{0, 0.1, 0.25, 0.6, 1\}$ is a five-level runtime-reachability estimate (not installed; installed with no dependency; transitively loaded but inactive; loaded with an unproven trigger; loaded with a demonstrated trigger). S uses ρ rather than a bare indicator because otherwise the label saturates at 100 once any affected CVE enters KEV and the positive control described below would not differ between its two conditions. A CVE is treated as active for image a at τ only if its publication date is $\leq \tau$ and its version-range status is `AFFECTED`, so no feature or label uses information not yet public at the time it purports to describe; y is left unlabelled (feature-only) when T extends beyond the last sealed evidence snapshot.

Evidence is sealed weekly: one EPSS bulk file per Monday and one CISA KEV catalogue download, each stored byte-for-byte and hashed (SHA-256) into a manifest that a verification step re-checks before any build; 57 weekly snapshots (2025-08-25 to 2026-09-21) and a KEV catalogue of 1,728 entries were sealed this way. Each image contributes one frozen, digest-pinned Trivy vulnerability scan; CVEs lacking a publication date (a known Debian/Ubuntu-tracker gap) are dated by the first sealed weekly snapshot in which they are scored, and a CVE that never appears in any

sealed snapshot is excluded from the active set rather than assumed absent. A positive control, P0-IF, isolates one asset with the image-filter Nginx module either inactive ($\rho = 0.25$: transitively present, unloaded) or active with its trigger path exercised ($\rho = 1$), built from the same digest-pinned base image as the primary asset P0 so that its vulnerability surface (109 of 135 known CVEs as of the freeze date, 25 in `libexpat`) is not an artefact of an unrelated rebuild.

At the time of writing the inventory holds 120 assets (117 images across web, proxy, cache, database, queue, runtime, base, observability, and Kubernetes-native infrastructure and security tooling categories, plus P0 and the two P0-IF conditions) across the 57-week window: 6,833 asset-week rows, after excluding seven rows whose evidence remained unresolved (one CockroachDB CVE under Trivy status `under_investigation` at the inventory freeze date) a non-missing rate of 0.93 and zero rows left in an unresolved evidence state. This clears the pre-registered data-quality gate (protocol frozen and dated; ≥ 10 observations per model parameter, i.e. $\geq 1{,}320$ rows for the 176-parameter configuration; no leakage across the split described below; every mandatory baseline runnable on the pilot), which we treat as a necessary condition for training HANFIS on real data rather than as a result in itself. The three open decisions on label aggregation, label scale, and the definition of Patch Risk were closed by cross-checking the decomposition above against this subsection's own formula rather than treated as free parameters: the row-wise maximum and the raw 0–100 scale are the definition as written, not a choice, and the $\{0,1,2\}$ Patch Risk contract already matches the running pipeline. The 120-asset inventory described here is the final inventory used for every result in Section 4; no further scaling of the inventory is planned for this paper.

3.6 Integration of Validated Telemetry with HANFIS

The proposed risk model contains seven conceptual inputs arranged hierarchically. The vulnerability subsystem receives CVSS, exploitability, and patch risk. The environmental subsystem receives MFA risk, Kubernetes exposure, and asset criticality. Threat intelligence is combined at the integration layer with the outputs of the vulnerability and environmental subsystems to produce the Dynamic Risk Score. Kubernetes telemetry principally contributes time-varying evidence to Kubernetes exposure and the broader runtime context; it does not replace the remaining six risk inputs.

Following data-quality and overhead assessment, eligible telemetry variables will be cleaned, temporally aligned, normalised, and mapped to interpretable runtime indicators. Candidate indicators include resource pressure, workload instability, pod or node availability, service degradation, and exposure-related deployment context. Feature selection must preserve semantic relevance to cyber risk: a metric is not included merely because it is available. The mapping from raw telemetry to an environmental risk input will be specified before final model evaluation to reduce post hoc feature selection.

Figure 1 summarises the relationship among infrastructure preparation, controlled telemetry acquisition, validation, and HANFIS. The figure is supplied as native LaTeX/TikZ so that it remains editable in the Springer manuscript.



Fig. 1 Methodological mind map linking the cloud-native testbed, telemetry acquisition, monitoring-overhead controls, reproducibility evidence, and HANFIS integration.

3.7 Validity Safeguards and Methodological Boundary

Internal validity rests on paired comparisons, a stable workload, condition gates, health checks, counterbalanced order, and the rejection of measurements taken under invalid states. Construct validity is protected by separating observability overhead from cybersecurity exposure and by requiring a semantic mapping before any telemetry variable is used as a HANFIS feature. Reliability is supported by script-based execution, frozen configuration and cryptographic integrity checks, and traceability runs from raw samples through summaries to model-ready features.

Several limits remain. WSL 2 with Docker Desktop is reproducible but does not represent every production Kubernetes deployment, and the two-node topology and

P0 workload constrain external validity. The two nodes together used on average about 230 mCPU without and 329 mCPU with the monitoring stack, i.e. 1.4% and 2.1% of the host's 16 logical cores (capacity from the v4.2 collector metadata and `docker info`; it was not logged during Runs 7–16), so the absence of a latency effect was observed at low utilisation and need not extend to saturated nodes. Ten pairs make an effect-size reading appropriate, but they bound the smallest attainable sign-flip p at 0.002 and give limited power for small latency effects. The protocol was frozen before the ten pairs were analysed, yet its version is entangled with the platform on which the earlier runs were taken, so cross-platform comparisons are descriptive. Finally, successful telemetry collection does not by itself show predictive value for cyber risk; that claim rests on the held-out HANFIS evaluation against the baselines of Section 4.

4 Results

4.1 Observability overhead

Ten complete run pairs passed every acceptance gate (Runs 7–16, protocol `external-v3.0`, counterbalanced five ON–OFF and five OFF–ON). One aborted attempt of Run 7 and one aborted attempt of Run 17 failed the integrity gate and were recorded as incidents rather than observations, in accordance with the fail-closed rule stated in Section 3. Table 3 reports the paired differences $\Delta Y_r = \bar{Y}_{r,\mathrm{ON}} - \bar{Y}_{r,\mathrm{OFF}}$.

Table 3 Paired monitoring overhead over ten counterbalanced runs (protocol external-v3.0). CI: 10,000-resample bootstrap; p: exact two-sided sign-flip test (floor $2/2^{10} = 0.002$).

Measure	Mean Δ	95% CI	Relative	p
Resource cost of the observability stack				
Node CPU (mCPU)	+99.36	[96.29, 103.35]	+43.3%	0.002
Monitoring namespace CPU (mCPU)	+39.79	[33.00, 52.77]	—	0.002
Node memory (MiB)	+280.76	[250.99, 303.54]	+18.7%	0.002
Monitoring namespace memory (MiB)	+204.19	[191.38, 218.82]	—	0.002
Effect on the observed workload				
Latency median (ms)	−0.005	[−0.055, +0.044]	−0.06%	0.859
Latency P95 (ms)	+0.059	[−0.126, +0.265]	+0.67%	0.607
Latency P99 (ms)	+0.188	[−0.127, +0.448]	+1.94%	0.268
P0 container CPU (mCPU)	−0.059	[−0.156, +0.041]	−3.9%	0.305
P0 container memory (MiB)	−0.334	[−0.522, −0.147]	−1.19%	0.016

Two results follow. First, the resource cost of the observability stack is large, consistent, and unambiguous: every pair shows the same direction and the effect sizes are extreme ($d_z = 16.5$ for node CPU). Node-level CPU increases by 99.4 mCPU while the monitoring namespace itself accounts for only 39.8 mCPU, so pod-level accounting understates the true cost by a factor of approximately 2.5; the remainder is induced platform work (kubelet, cAdvisor, container runtime, and the API server serving the additional scrape traffic). The same pattern holds for memory (280.8 MiB at node level against 204.2 MiB inside the namespace).

Second, this resource cost does not propagate to the observed workload. For median latency and P95 latency we fail to reject the null hypothesis, with confidence intervals of $[-0.06, +0.04]$ ms and $[-0.13, +0.27]$ ms respectively, that is, below 3% of the baseline latency. The P0 container consumes marginally *less* memory under monitoring (-0.33 MiB, $p = 0.016$), an effect of practical size zero that we report for completeness rather than interpret. With ten pairs the sign-flip test can reach $p = 0.002$, so the failure to reject on latency reflects the absence of a detectable effect rather than the absence of statistical power.

These measurements justify treating the observability stack as a fixed resource overhead that must be budgeted for, and justify using the collected telemetry as HANFIS features without correcting for workload latency due to instrumentation effects. We do not claim equivalence: the confidence intervals bound the effect; they do not exclude it.

4.2 Training configuration

Throughout, "pre-registered" means that a dated protocol was committed to the project's version-controlled repository and tagged before the corresponding training run; it does not refer to an entry in an external registry. The confirmatory protocol was committed on 30 September 2026 and tagged the same day (tag `e4-prereg-2026-09-29`, whose name carries the nominal date of the protocol file), and the first confirmatory training run took place on 2 October 2026.

All results below use the frozen configuration of amendment A-09: hybrid closed-form/Adam training (Section 3.2), learning rate 10^{-2}, ridge 10^{-3}, three pre-registered restarts per fold with fixed seeds, patience 100 on validation loss with a minimum of 200 epochs, the Gaussian membership family with the monotonicity constraint active (justified in Section 4.5). Evaluation uses a grouped, embargoed temporal split (`GroupTimeSplit`, amendment A-16): images are assigned to n held-out groups by a seeded permutation, fold k trains on every other group up to a cutoff date and evaluates group k inside the following block of weeks after a 30-day embargo equal to the outcome window, so no image and no overlapping outcome window appears on both sides of any fold; every image is evaluated exactly once. On the completed 120-asset inventory this yields 5 folds of 95–96 training images against 23–24 held-out images. The alternating lower-panel closed-form training variant (Section 3.2) was verified against an independent closed-form derivation on synthetic data (mean squared error $0.78 \to 4 \times 10^{-4}$ over six alternating iterations with no premise training at all) before being adopted as amendment A-09's frozen training scheme, used throughout the real-data results reported below and in Section 4.4.

4.3 Predictive performance

Table 4 reports the mandatory baselines on the 6,357 labelled rows described in Section 3.5, pooled over all five test folds (one mean-squared-error ratio over every held-out row, rather than an average of per-fold ratios, which is unstable when a fold's reference error is near zero). $\mathrm{SS}_{\mathrm{clim}}$ and $\mathrm{SS}_{\mathrm{pers}}$ are skill scores against the training-fold climatology and against a causal 4-week persistence forecast, respectively.

Table 4 Mandatory baselines, pooled over 5 grouped-temporal folds, raw label scale (0–100).

Baseline	RMSE	SS_{clim}	SS_{pers}
Climatology	4.010	0.000	−37.78
Single CVSS score	3.537	0.222	−29.17
Persistence (causal, 4-week lag)	0.644	0.974	0.000
Ridge regression, all 7 inputs	1.169	0.915	−2.293

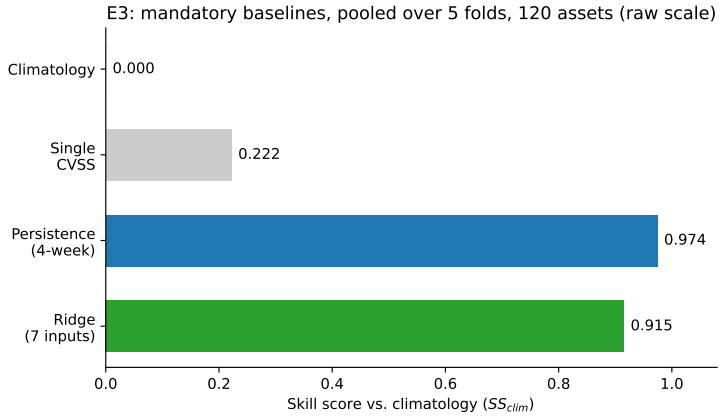


Fig. 2 Mandatory baselines' skill against climatology, pooled over 5 folds. A single CVSS score now adds modest skill over climatology (22%, up from essentially nothing on the earlier 27-image pilot); persistence and ridge separately recover over 90% of the climatology-relative skill each, though not of each other – see Table 4 for the reversal in their relative standing.

A single CVSS score explains a modest fraction of the realised label's variance ($SS_{clim} = 0.222$) – markedly more than on the earlier 27-image pilot ($SS_{clim} \approx 0$), though still far below persistence or ridge – confirming that static severity alone remains a poor descriptor of which vulnerabilities become exploitable in practice, even as the larger inventory gives it some traction. A naive persistence forecast, in contrast, recovers 97% of the climatology-relative skill: the label is strongly autocorrelated because the set of active CVEs and their EPSS scores change slowly week to week. On the completed 120-asset inventory, a linear ridge model over all seven inputs no longer improves on persistence – it is now substantially worse ($SS_{pers} = -2.293$ pooled, raw scale), a reversal from the 27-image pilot's +0.301, confirmed on every one of the five grouped-temporal folds rather than driven by one outlier (Section 5.1). This is consistent with a linear model over-extrapolating as the inventory's heterogeneity grew, where persistence's week-to-week copy remains comparatively safe. Persistence is therefore the bar hypotheses H1b and H1c (Section 1) ask HANFIS to clear or match; ridge alone no longer clears it either. The HANFIS-versus-persistence, HANFIS-versus-ridge, and HANFIS-versus-FlatANFISMatched comparisons (H1a–H1c, H2, H3) are

pre-registered under a frozen protocol (fixed seeds, fold assignment, smallest-effect-size-of-interest of 5% on SS_{pers}, group-clustered bootstrap confidence intervals, Holm correction across the family of four comparisons). Table 5 reports the outcome on the 120-asset inventory described in Section 3.5: HANFIS confirms H1a, H1c, and H2, does not yet confirm H1b, and H2′ is reported as the uncriteria'd capacity reference. Fuzzyfortify is not part of the statistical family: it requires four native inputs (complexity-resilience index, impact, base score, exploitability) that the asset-week data do not produce, and the comparison protocol forbids unreported proxy mappings. We therefore neither use it as a baseline nor report a Fuzzyfortify comparison.

Table 5 E4 confirmatory results: HANFIS skill score against each comparator, group-clustered bootstrap 95% CI, Holm-corrected across the four-hypothesis family.

Hypothesis	Comparator	SS	95% CI	Outcome
H1a	Single CVSS score	0.967	[0.951, 0.980]	Confirmed
H1b	Persistence (4-week)	0.015	[−0.607, 0.246]	Not yet confirmed
H1c	Ridge, 7 inputs	0.696	[0.527, 0.816]	Confirmed
H2	FlatANFISMatched	0.665	[0.269, 0.844]	Confirmed
H2′	Flat ANFIS (full)	0.253	—	Reported, no criterion

H1b's confidence interval crosses zero, so HANFIS is not yet distinguishable from a causal 4-week persistence forecast – the same pattern observed in the E3 pilot (Section 4.3's baseline table above), and the honest reading is that the strongly autocorrelated label remains the hardest bar in this study. The post hoc analysis in Section 4.5 indicates why: HANFIS outperforms persistence when the label moves and underperforms it when the label is stable. H2's margin is large for two models holding an identical 176-parameter budget, though not uniform across folds: HANFIS wins on four of the five held-out blocks, with per-fold skill against FlatANFISMatched ranging from 0.165 to 0.924; the fifth fold (fold 1) is negative (SS = −0.122) and coincides with a secondary diagnostic in which HANFIS's predicted variance exceeds the label's own variance by more than three orders of magnitude on several near-constant-label images in that fold (ratio ¿1,000× in the most extreme case), consistent with the model under-damping day-to-day EPSS/CVSS snapshot noise where the weekly aggregate carries little real signal – a limitation discussed further in Section 5.1, not a single outlier fold driving an otherwise uniform result. FlatANFISMatched's held-out predictions nonetheless range well outside the plausible label interval (down to −6.679 and up to 11.059, against HANFIS's 0.000–10.401) in a manner consistent with poor extrapolation from unsupervised cluster centres that have no access to the label's own generative grouping – precisely the vulnerability/environmental/threat decomposition HANFIS's architecture encodes by construction (Section 3.2). This is read as evidence for the mechanism claimed in C1/H2 rather than a methodological artefact: the random 15% validation carve-out used only to select the best of three restarts is identical for both models, while the reported skill score itself comes from the temporally and group-disjoint held-out block that carve-out never touches.

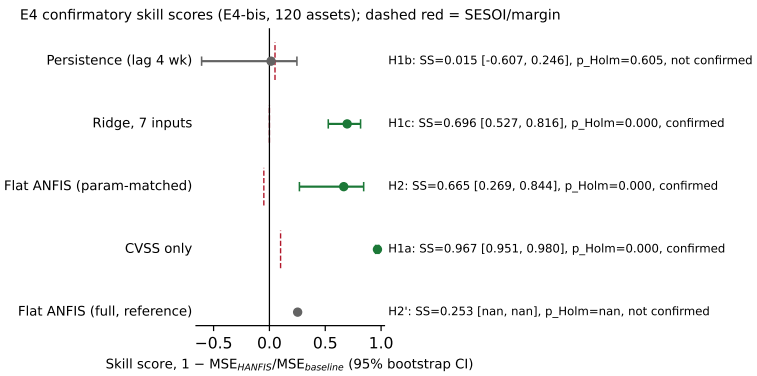


Fig. 3 E4 confirmatory results: HANFIS skill score against each comparator with 95% group-clustered bootstrap confidence intervals. Green bars exclude zero (confirmed); the red bar (H1b, vs. persistence) does not.

4.4 Accuracy-complexity curve

The scalar inter-tier bottleneck between the lower panels and the integration panel is a candidate explanation for any residual gap between HANFIS and a full-capacity flat ANFIS: compressing v and e to a single number may discard cross-subsystem interaction that the flat model can still represent. A sweep over the bottleneck width k – widening v and e from scalars to k-dimensional vectors passed into the integration panel – traces this trade-off directly, but grid-partitioning the integration tier makes its rule count grow as $m_I^{2k} \cdot m$: at the frozen $m_I = 3$, $k = 1$ reproduces HANFIS exactly (34 rules, 176 parameters), $k = 2$ already costs 178 rules and 1,152 parameters (6.5x), and $k \geq 3$ exceeds 1,400 rules (Figure 4) – more parameters than the present inventory has rows, and the sweep script refuses these by default rather than attempting an underdetermined fit. $k \in \{1, 2\}$ were run; $k = 4$, originally planned, was dropped on these grounds and this combinatorial explosion is itself read as evidence for why a scalar bottleneck was the design choice in the first place, independent of what $k = 1$ vs. $k = 2$ shows empirically.

At k=1 the sweep uses exactly A-09's training scheme – the alternating closed-form least squares for both lower panels, the intercept refit after the monotone projection, and the same hard clamp on v, e that HANFIS's own forward pass applies – implemented separately in `hanfis_bottleneck.py` under a multi-head representation that generalises cleanly to $k > 1$. A per-row comparison against the k=1 predictions underlying Table 5 confirms this reproduction directly rather than merely asserting it: Pearson $r = 0.9984$ on point predictions, and identical fold assignment for all 981 held-out rows. An earlier version of this sweep used a simpler training scheme (the Integration-only closed form rather than the alternating scheme) and omitted the intercept refit after the monotone projection; both have since been corrected, and the numbers below supersede that earlier, confounded comparison.

k=1 recovers skill broadly consistent with HANFIS's own H1b result (Table 5: $\mathrm{SS}_{\mathrm{pers}} = 0.015$, $[-0.607, 0.246]$) – both confidence intervals cross zero and overlap substantially, as the per-row parity above predicts, though the point estimates

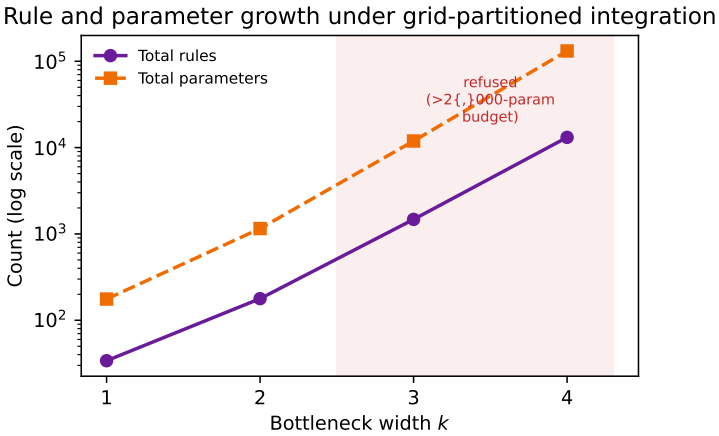


Fig. 4 Rule and parameter growth with bottleneck width k under grid partition of the integration tier. The shaded region ($k \geq 3$) exceeds the parameter budget the present inventory supports and was refused rather than attempted.

Table 6 E5 bottleneck-width sweep: pooled RMSE and skill against persistence, 95% confidence intervals from 2,000 image-clustered bootstrap resamples (the same procedure as Table 5), 120-asset inventory.

k	Rules / Params	RMSE (pooled)	$\mathrm{SS}_{\mathrm{pers}}$ [95% CI]
1	34 / 176	0.587	0.169 [−0.330, 0.343]
2	178 / 1,152	0.820	−0.622 [−2.259, −0.100]

(0.169 vs. 0.015) agree less closely than on the smaller inventory. k=2's interval now excludes zero entirely on the negative side: on the completed 120-asset inventory, widening the bottleneck is no longer merely unsupported by evidence – it is significantly worse than persistence. The paired comparison on matched resampled images – the same resample used for both arms, so sampling noise is shared – gives $\mathrm{SS}_{\mathrm{pers}}(k{=}1) - \mathrm{SS}_{\mathrm{pers}}(k{=}2) = 0.791$, 95% CI $[0.354, 2.037]$, excluding zero (100% of resamples favour k=1); k=1 is pooled-RMSE-better on all five folds (two-sided sign test $p = 0.0625$, the floor attainable with five folds) – the same margin of evidence as on the smaller inventory, even as each arm's marginal significance against persistence flipped in opposite directions.

This is the strongest statement the present inventory supports, and it remains a mechanism claim rather than a claim that wider bottlenecks are intrinsically worse: every k=2 fold falls under ten observations per parameter (range 0.3–2.9), the same ten-per-parameter threshold applied elsewhere in this study (Section 3.5), while k=1 clears it in three of five folds. A secondary diagnostic separates part of this gap from pure data scarcity: disabling the hard clamp on v, e (an ablation, not the model HANFIS reports) costs k=1 0.161 of skill but costs k=2 0.813 (both re-run on the completed

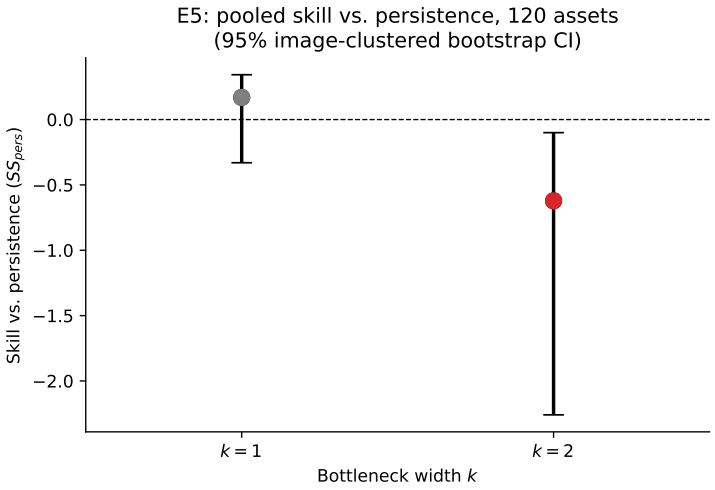


Fig. 5 E5: pooled skill against persistence for $k \in \{1, 2\}$ with 95% image-clustered bootstrap confidence intervals. k=1's interval now crosses zero; k=2's is entirely negative.

120-asset inventory; up from 0.019 and 0.394 respectively on the 27-image pilot) – consistent with v, e being estimated less reliably from the same training rows once six times as many consequent parameters compete for them, so that excursions outside $[0, 1]$ are both more frequent and more consequential once clamped. Separating an intrinsic cost of wider bottlenecks from this data-scale effect requires an inventory substantially larger than the 120 assets collected here, left for future work; the architecture's own combinatorial growth in k (Figure 4) means that separation gets harder, not easier, as k increases.

4.5 Ablation and Robustness Analysis

Table 7 reports the completed membership-family×monotonicity ablation (E1) on two synthetic scenarios designed to stress-test the training scheme before it is committed to real data: S1, a smooth response with no thresholds, and S2, a response containing a hard threshold, each fit with five seeds at the 176-parameter (Gaussian) or 147-parameter (Ruspini) configuration.

The monotonicity constraint costs little on the smooth scenario but is expensive on the threshold scenario for the Gaussian family specifically (RMSE 0.117 → 6.067), because a hard threshold is locally non-monotone and softplus-constrained slopes cannot represent the local reversal; the connected Ruspini family degrades far less under the same constraint (0.646 → 2.126) because its partition of unity concentrates the approximation error near the threshold rather than distributing a large penalty across every rule. Because the real target in Section 3.5 is not known in advance to be smooth or threshold-like, amendment A-10 retains the Gaussian family with monotonicity active as the frozen default (consistent with the higher-is-riskier requirement) and treats Ruspini as a documented, cheaper fallback if the monotone-Gaussian cost proves large on real folds. An end-to-end pure-Adam variant without the closed-form

Table 7 E1 ablation: membership family × monotonicity, 5 seeds per cell.

Scenario	MF family	Monotone	Params.	RMSE (mean ± SD)	R^2 (mean)
S1 (smooth)	Gauss	no	176	0.093 ± 0.021	1.000
S1 (smooth)	Gauss	yes	176	0.411 ± 0.246	0.999
S1 (smooth)	Ruspini	no	147	0.043 ± 0.052	1.000
S1 (smooth)	Ruspini	yes	147	0.073 ± 0.066	1.000
S2 (threshold)	Gauss	no	176	0.117 ± 0.034	1.000
S2 (threshold)	Gauss	yes	176	6.067 ± 1.524	0.879
S2 (threshold)	Ruspini	no	147	0.646 ± 0.051	0.999
S2 (threshold)	Ruspini	yes	147	2.126 ± 0.906	0.983

anchor (Section 3.2) was also tested during this ablation and diverged on a subset of seeds under otherwise identical settings, confirming the hybrid scheme as necessary rather than incidental.

Sensitivity to the EPSS model change. The EPSS scoring model changed version on 2026-06-15 (`v2025.03.14` to `v2026.06.15`). Because the label of week t is computed from snapshots $t{+}7, \ldots, t{+}28$ days ahead, two of the five held-out blocks of the confirmatory comparison involve labels computed under the newer version: one block straddles the change and one lies wholly after it. This contrast was not pre-registered, so it is reported as a post hoc sensitivity analysis. The confirmatory pipeline was re-run with the same code, seeds and frozen configuration on the 4,560 asset-weeks whose labels all pre-date the change (weeks up to 2026-05-11; 720 held-out rows), with the results in Table 8. H1a, H1c and H2 remain confirmed, and H1b remains unconfirmed, with a much wider interval. Because both the training and the held-out windows change, skill scores are not directly comparable across the two columns; the larger margin over the matched flat model (H2: 0.665 to 0.913) comes from the matched model, whose mean squared error per held-out row rises from 1.22 to 3.77 while that of HANFIS stays at 0.41 and 0.33.

Table 8 Post hoc sensitivity of the confirmatory comparison to the EPSS model change: full run versus weeks whose labels all pre-date 2026-06-15 (skill score, 95% group-clustered bootstrap CI).

Hypothesis	Comparator	Full run	Pre-change weeks
H1a	Single CVSS score	0.967 [0.951, 0.980]	0.973 [0.939, 0.991]
H1b	Persistence (4-week)	0.015 [−0.607, 0.246]	−0.055 [−2.731, 0.201]
H1c	Ridge, 7 inputs	0.696 [0.527, 0.816]	0.728 [0.484, 0.901]
H2	FlatANFISMatched	0.665 [0.269, 0.844]	0.913 [0.753, 0.974]
H2′	Flat ANFIS (full)	0.253	0.441

Where HANFIS and persistence differ. The pooled skill against persistence (0.015) hides two regimes, and this exploratory, post hoc analysis (not part of the Holm-corrected family) separates them. Dropping one held-out fold at a time leaves it

between 0.007 and 0.172 except for fold 3, whose removal gives −1.168. Fold 3 ends at the EPSS model change, and the label of week t is built from snapshots up to t+28 days ahead whereas the persistence forecast reuses the label of week t−4. For the four weeks in which the label is fully computed under the new model while the lagged label is still computed under the old one (2026-W24 to W27; 95 rows), skill against persistence is +0.483; on the remaining 985 rows it is −0.420. In 2026-W25 and W26 the squared error of persistence is 63.1 and 28.9 against 0.7 and 1.0 for HANFIS, while the parameter-matched model also adapts (4.8 and 5.2), so the advantage reflects access to the current EPSS input rather than the hierarchical structure. In the week the label first jumps (W24) neither HANFIS nor persistence anticipates it (108.9 and 115.2). In the stable weeks that follow HANFIS keeps a small error floor (about 0.7–0.8 per week over 24 rows, against 0.1 for persistence). The pooled advantage of 6.5 squared-error units is thus the sum of +104.1 in the break weeks and −97.6 elsewhere. The same pattern appears before the change: in the run restricted to earlier weeks the advantage concentrates in 2026-W06 and W07, the weeks of one image's label surge (an image carrying 77.8% of the persistence error), and the total is −12.3. HANFIS is therefore reactive rather than predictive: better than persistence when the label moves, worse when it does not. Two further observations qualify H2. The first held-out fold, trained on 2.2 observations per parameter, contributes 91% of the pooled gap to the matched model (squared error 869.6 against 73.9); without it the skill against FlatANFISMatched is 0.183 and against the full flat model 0.084, whereas the skill against the ridge model stays between 0.656 and 0.793 whichever fold is removed.

4.6 Explanation fidelity (E6)

We tested whether the rule-level explanation that HANFIS exposes is faithful to its inference. The test uses the 25 saved models (five folds × five seeds), which reproduce the confirmatory predictions of Section 4 exactly (mean over the five seeds; maximum difference 1.4×10^{-6} on the 0–100 scale in single precision). The hypotheses and thresholds were drafted and announced before execution; the version-control freeze, however, preceded the commit that contained the protocol text, so the protocol is not frozen in the strict sense and the run is reported as a documented deviation. Table 9 lists the outcome.

Removing the three rules ranked highest by attribution changed the output 4.1 times more than removing three random rules, but it exceeded the median random removal in only 83.5% of non-degenerate rows (criterion: 90%); 456 of 5,400 rows (8.4%) were degenerate, 446 of them because the output was clamped. Of the 575 positive-control pairs, 552 increased strictly; the remaining 23 include 20 pairs whose output is clamped at zero in both conditions. Post hoc and without bearing on the decision, the proportion reached 0.90 in two of five folds and was lowest in fold 3 (0.637), and it was lowest on the P0 and P0-IF rows (0.648, ratio 1.73, 135 rows). On the nine paired test weeks the mean ON–OFF contrast was 0.037 for the model and 0.073 for the label (0–100 scale): the direction is right and the magnitude is about half.

We therefore do not claim validated rule-level explanations. The rule trace matches the inference exactly and the direction of the response is verified, but the attribution of the output to individual rules is faithful on average rather than uniformly.

Table 9 Explanation-fidelity checks on the 25 saved models. Pairs are the positive-control images P0-IF with and without the active filter; S^+ denotes the 23 weeks in which exploitability differs between the two.

Check	Criterion	Result	Outcome
H6-0 trace vs. inference	$\lvert\Delta\rvert \leq 10^{-9}$, all test rows	0	pass
H6-neg identical inputs	$\lvert\Delta\rvert \leq 10^{-9}$, 34 weeks × 25 models	0	pass
H6a direction (monotone)	$\hat{y}_{\text{on}} - \hat{y}_{\text{off}} \geq -10^{-9}$ on S^+	minimum 0	pass
H6b panel structure	panel E unchanged, panel V changes	0 and 2.9×10^{-4}	pass
H6c deletion test	ratio ≥ 2 and proportion ≥ 0.90	4.11 and 0.835	fail

4.7 Attribution and monotonicity checks (E8)

We computed exact interventional Shapley values for the seven inputs of the 25 saved models (5,400 model–row pairs; a background of 60 random labelled rows, excluding P0 and P0-IF and identical for all models; the efficiency property holds to 2×10^{-14}). Exploitability (x_2) and the KEV indicator (x_7), both ingredients of the label, carry 78.9% of the mean absolute attribution; the three environment inputs (x_4–x_6) carry 3.3% (Table 10). The ranking is stable across models (Kendall $W = 0.79$; x_2 is the top input in 21 of 25 models), and in the 20% of rows with the highest label x_7 alone carries 63.0%. This quantifies the overlap between label and inputs discussed in Section 5: the model largely forecasts exploitability signals it can already see, and the label contains no environment component by construction.

Table 10 Exact Shapley attribution and within-row monotonicity checks for the five-seed ensemble (E8; descriptive and post hoc). "Falls" is the share of steps, over the range observed in the data, in which the score drops by more than 0.05 points when one input is raised and the others are held fixed. Share: mean absolute attribution; Max drop and Range: largest fall and mean end-to-end change of the score in points.

Input	Share (%)	Falls (%)	Max drop	Range
x_1 CVSS	10.9	1.7	2.46	1.99
x_2 Exploitability	46.4	1.2	0.21	6.53
x_3 Patch Risk	6.8	24.8	2.28	1.11
x_4 MFA Risk	1.1	29.1	0.46	0.09
x_5 Kubernetes Exposure	1.3	34.8	0.60	0.14
x_6 Asset Criticality	0.9	10.5	0.40	0.13
x_7 Threat Intelligence (KEV)	32.5	1.2	0.33	6.45

Non-negative rule slopes do not guarantee a monotone score, because rule weights are normalised and intercepts are free. We therefore varied one input at a time over 11 points spanning the observed range (observed values for discrete inputs, quantiles from 1% to 99% otherwise) for the five-seed ensemble that is reported in Section 4. For the dominant drivers the share of falling steps is small (x_1 1.7%, x_2 1.2%, x_7

1.2%), whereas for patch risk and the environment inputs it is 10–35%, although the environment inputs move the score by less than 0.15 points on average (Table 10). Individual seeds are less well behaved than the ensemble: the largest single drop (11.6 points, CVSS) occurs in one seed on two assets in the last held-out block, after the EPSS model change. We therefore present monotonicity as an empirical property of the dominant inputs, not as a guarantee of the architecture.

5 Discussion

Three findings from the completed experiments (E1, E3, and the observability-overhead runs) shape how the confirmatory comparison in Section 4.3 should be read. First, the label is dominated by its own recent history: a 4-week persistence forecast recovers 96% of the climatology-relative skill on the pilot inventory, so any claim that HANFIS "predicts risk" must be a claim about improving on a strong autoregressive baseline, not about beating an uninformative one – this is why H1 was reframed against persistence rather than against Fuzzyfortify or CVSS alone (Section 1). Second, a single CVSS score carries essentially no explanatory power for the realised 30-day exploitability exposure ($SS_{clim} \approx 0$), which is itself evidence for the research gap C3 claims to address: severity and realised exploitation are different quantities, and conflating them is a plausible source of alert fatigue in practice. Third, the E1 ablation shows that a monotonicity constraint intended to keep the model auditable (higher risk factors must not lower the score) has a real, family-dependent accuracy cost on data containing thresholds; the choice of Gaussian membership functions with monotonicity active is therefore a deliberate trade of some accuracy for a response that, for the dominant risk factors, rarely falls as they rise (Section 4.7), not a default that happened to be convenient.

The instrumentation-overhead results (Section 4.1) support a second, narrower claim: that Kubernetes observability telemetry can be treated as a fixed resource cost rather than folded into the risk model as a confound. Node-level CPU increases by 99.4 mCPU under monitoring while the monitoring namespace itself accounts for only 39.8 mCPU, so pod-level accounting alone understates the true platform cost by a factor of approximately 2.5; despite this, workload latency shows no detectable change (95% confidence intervals within ± 0.27 ms). This separates the cost of observing the system from any change in the thing being observed, which is the premise the environmental subsystem's Kubernetes-exposure input relies on.

5.1 Limitations

Two limits of construct validity deserve emphasis before the usual caveats. First, the label and two of the inputs share ingredients: x_2 (EPSS $\times \rho$) and x_7 (the KEV indicator) are the values at t of quantities that the label aggregates over $t{+}7, \ldots, t{+}28$, and runtime reachability ρ enters both. The task is therefore to forecast the near future of exploitability signals that the model can already see, not to predict independent compromise outcomes, for which no ground truth exists here; the persistence baseline, built in the same way, is the relevant bar, and the comparison with a single CVSS score should be read in that light; exact Shapley attribution (Section 4.7) places 78.9%

of the model's attribution on these two inputs and 3.3% on the environment inputs. Second, the dynamic character of the score comes from the time-varying EPSS and KEV feeds and from runtime reachability; runtime telemetry (CPU, memory, latency) is used only to quantify observability overhead and is not a model input, so "dynamic" must not be read as telemetry-driven scoring. The rule-level attribution is faithful on average but not uniformly (Section 4.6), and we do not claim validated explanations.

The confirmatory comparison (Table 5) uses the full 120-asset inventory. On the same inventory (6,357 labelled rows; Table 4), a linear ridge model over all seven inputs underperforms persistence on every one of the five grouped-temporal folds ($SS_{pers} = -2.293$ pooled, raw scale); we suspect that the linear model over-extrapolates as the inventory's heterogeneity grows, where persistence's week-to-week copy remains comparatively safe, but we did not test this explanation. The EPSS scoring model changed version (`v2025.03.14` to `v2026.06.15`) inside the sealed 57-week window, a threat to label comparability across that boundary; two of the five held-out blocks involve labels computed under the newer version, and a post hoc re-analysis restricted to weeks whose labels all pre-date the change leaves the confirmatory conclusions unchanged (Section 4.5). The data-sufficiency gate (at least ten observations per parameter) is met by the pooled sample (6,357 labelled rows against 176 parameters) but not by the two earliest held-out folds (2.2 and 7.1 observations per parameter), which are also those in which HANFIS is furthest behind persistence, and H2's pooled margin is dominated by the first fold (Section 4.5). CVEs lacking a Trivy-reported publication date are dated by their first appearance in a sealed EPSS snapshot, which is conservative (at most 7 days late) but not exact. The label aggregation rule (a row-wise maximum over active CVEs), its numeric scale, and the operational definition of Patch Risk were closed and pre-registered as amendment A-13 (cross-checked against the literal target-definition formula, Section 3.5) before the confirmatory comparison in Table 5 was run. The testbed is a single two-node WSL 2/Docker Desktop Kubernetes cluster; external validity to production multi-node clusters and other CNI/storage stacks is untested. Ten paired observability runs, while showing large and consistent effects, still provide limited power for null-hypothesis testing on the latency measure specifically, which is why the reported confidence intervals rather than a significance threshold carry the interpretive weight in Section 4.1.

6 Conclusion and Future Work

This paper proposes HANFIS, a domain-informed hierarchical ANFIS that reduces a 128-rule, 1,052-parameter flat Sugeno system over seven cloud-native risk inputs to 34 rules and 176 parameters (Section 3.2) by separating vulnerability and environmental reasoning into two first-tier panels before an integration tier combines them with dynamic threat intelligence. Under a protocol frozen and dated before training began (Table 5), HANFIS confirms a substantial reduction in estimation error against a single CVSS score (H1a), a linear model over the same seven inputs (H1c), and a parameter-matched flat ANFIS initialised by subtractive clustering (H2); it is not yet distinguishable from a causal 4-week persistence forecast (H1b), which this study's own mandatory baselines (Section 4.3) show to be a far harder bar than static

expert scoring, not an afterthought comparator. H2 is confirmed as non-inferiority to a capacity-matched flat model, and HANFIS has the lower error in four of the five held-out blocks (Section 4.3); the pooled margin is, however, dominated by the first block, where training is thinnest, so it is read as suggestive of, not as evidence for, a structural advantage of the vulnerability/environmental/threat decomposition (Section 4.5). Two further contributions are established: the real-world data pipeline itself (sealed, hash-verified EPSS and CISA KEV evidence feeding a gated inventory of 120 assets and 6,357 labelled asset-weeks, Section 3.5), and a completed ablation showing that the monotonicity constraint intended to keep the score from falling as risk factors rise (met in practice only for the dominant drivers; Section 4.7) has a real, membership-family-dependent accuracy cost (Section 4.5).

Confirming H1b against persistence is the main open question; the bottleneck-width sweep (Section 4.4) found that widening the scalar inter-tier bottleneck from k=1 to k=2 does not close the gap to persistence and is significantly worse than it. Re-scanning every image with a single frozen vulnerability database, so that no image's CVE surface differs merely because it was scanned on a different day, remains an open methodological item but is not expected to change the qualitative picture given the gate margin already achieved. The auditable policy-translation layer (contribution C4) and edge deployment (originally hypothesis H5, now future work since the present testbed runs HANFIS cluster-side rather than on its edge node) remain designed but unevaluated, and are left for a subsequent study once the predictive comparison above is closed.

Acknowledgements

This work is supported by the Institute of Research and Community Service, Universitas Andalas (UNAND), Indonesia.

Declarations

- **Funding.** This work is supported by the Penelitian Kepakaran research grant, Institute for Research and Community Service (LPPM), Universitas Andalas, Indonesia.
- **Conflict of interest/Competing interests.** The authors declare no competing interests.
- **Ethics approval and consent to participate.** Not applicable: this study involves no human participants, human data, or animal subjects. All measurements were collected from an isolated Kubernetes research testbed operated by the authors.
- **Consent for publication.** Not applicable.
- **Data availability.** The sealed EPSS/KEV evidence manifests, container-image vulnerability scan bundles, and derived asset-week tables are maintained in a private repository during the study and will be made available upon publication, subject to the licensing terms of the underlying EPSS and CISA KEV feeds.
- **Materials availability.** Not applicable.

- **Code availability.** The HANFIS implementation, data pipeline, and experiment scripts are maintained at `udahafeez7/HANFIS-RESEARCH` (private during review) and will be made available upon publication.
- **Author contribution.** M.H.H.: conceptualisation, methodology, software, experimentation, writing – original draft. D.D. and B.R.: supervision, methodology review, writing – review and editing.
- **Use of AI tools.** During the preparation of this work the authors used an AI assistant (Claude, Anthropic) to help write and test analysis and utility code, to draft and revise manuscript text, and to review study protocols. The authors ran and verified all code and results, reviewed and edited all text, and take full responsibility for the content of the publication.

References

[1] Chandramouli, R.: Security strategies for microservices-based application systems. NIST Special Publication 800-204, National Institute of Standards and Technology (2019). https://doi.org/10.6028/NIST.SP.800-204

[2] The Kubernetes Authors: Pod Lifecycle. https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/. Accessed 6 October 2026 (2026)

[3] Rose, S., Borchert, O., Mitchell, S., Connelly, S.: Zero trust architecture. NIST Special Publication 800-207, National Institute of Standards and Technology (2020). https://doi.org/10.6028/NIST.SP.800-207

[4] Chandramouli, R., Butcher, Z.: A zero trust architecture model for access control in cloud-native applications in multi-location environments. NIST Special Publication 800-207A, National Institute of Standards and Technology (2023). https://doi.org/10.6028/NIST.SP.800-207A

[5] Hafiz Hersyah, M., Hossain, M.D., Taenaka, Y., Kadobayashi, Y.: Fuzzyfortify: a multi-attribute risk assessment for multi-factor authentication and cloud container orchestration. Frontiers in Computer Science **7**, 1557918 (2025) https://doi.org/10.3389/fcomp.2025.1557918

[6] Jang, J.-S.R.: ANFIS: Adaptive-network-based fuzzy inference system. IEEE Transactions on Systems, Man, and Cybernetics **23**(3), 665–685 (1993) https://doi.org/10.1109/21.256541

[7] Joint Task Force Transformation Initiative: Guide for conducting risk assessments. NIST Special Publication 800-30 Rev. 1, National Institute of Standards and Technology (2012). https://doi.org/10.6028/NIST.SP.800-30r1

[8] FIRST.org, Inc.: Common Vulnerability Scoring System Version 4.0: Specification Document. https://www.first.org/cvss/v4.0/specification-document. Accessed 6 October 2026 (2023)

[9] Jacobs, J., Romanosky, S., Edwards, B., Roytman, M., Adjerid, I.: Exploit prediction scoring system (EPSS). Digital Threats: Research and Practice **2**(3), 1–17 (2021) https://doi.org/10.1145/3436242

[10] Wang, L.-X.: Universal approximation by hierarchical fuzzy systems. Fuzzy Sets and Systems **93**(2), 223–230 (1998) https://doi.org/10.1016/S0165-0114(96)00197-2

[11] Ojha, V., *et al.*: Heuristic design of fuzzy inference systems: A review of three decades of research. Engineering Applications of Artificial Intelligence (2019) https://doi.org/10.1016/j.engappai.2019.08.010

[12] Lee, M.-L., Chung, Yu: Modeling of hierarchical fuzzy systems. Fuzzy Sets and Systems (2003) https://doi.org/10.1016/S0165-0114(02)00517-1

[13] Raju, G.V.S., Zhou, J., Kisner, R.A.: Hierarchical fuzzy control. International Journal of Control **54**(5), 1201–1216 (1991) https://doi.org/10.1080/00207179108934205